Commande du SLIDER en Python

Configuration

- Boitier Slider connecté au réseau par câble RJ45 ou wifi
- Boitier Slider connecté au PC par câble USB

Le numéro du port série est accessible via le panneau de configuration.

Protocole de communication

Le protocole de communication consiste à envoyer à l'Arduino une suite d'octets (bytes en anglais) correspondant à une commande. L'Arduino répond à cette commande en écrivant sur la liaison série une suite d'octets.

La conversion d'une commande écrite sous le format chaines de caractères (string en anglais) en une suite d'octects se fait de façon transparente en ajoutant le préfixe b devant la chaîne de caractères. Exemple `b"?#"`.

Toutes les commandes se terminent par le caractère #.

La communication se fait à 115200 bauds sur 8 bits, sans parité avec 1 bit de stop.

Le premier test à faire consiste à envoyer "?#". Le Slider doit répondre "ok". Ce message signifie que la configuration est bonne.

Communication série à partir de l'IDE Python

La communication série entre l'IDE python et le SLIDER nécessite la bibliothèque `serial`.

L'ouverture de la liaison série entre le PC et l'Arduino prend quelques millisecondes. Elle nécessite une pause dans le programme python et donc l'importation de la bibliothèque `time`.

Programme test qui retourne "ok" si la communication est fonctionnelle (Communication_serie_python_1.py)

```
import time
from serial import *

# ouverture de la liaison série
ps=Serial(port="COM7",baudrate=115200)
time.sleep(0.1) # pause permettant l'établissement de la liaison série

# écriture de la commande sur la liaison série
ps.write(b"?#")

# lecture de la réponse de l'Arduino
reponse=(ps.read(2))

# transtypage byte -> str
reponse=reponse.decode("utf-8")

# affichage de la réponse
print(reponse)

# fermeture de la liaison série
ps.close()
```

Liste des commandes de contrôle

?# : retourne "ok", pour test de communication

V# : retourne la version du logiciel

E# : code de fin d'une série de commandes (end), retourne un retour à la ligne

Liste des commandes d'exécution

`O#` : demande de prise d'origine. Arduino retourne 2 octets ("OK")

<u>La commande de prise d'origine doit être exécutée avant toute commande de translation du rail.</u>

`T0.XXX/0.XXX#` : demande de translation du rail, vitesse et position (13 caractères au total)

Ordre T, vitesse en mètre/seconde au format 0.xxx, distance en mètre au format 0.xxx

Vitesse maximale 0.2m/s, nécessite une prise d'origine

Exemple de commande en translation : `T0.125/0.453#` (vitesse 0.125 m/s position 0.453mm)

`Rxx.x/+y.yy#` : demande de rotation de la tête, vitesse xx.x, angle y.yy (12 caractères au total)

Ordre R, vitesse en radian/seconde au format xx.x, angle en radian au format +/-x.xx (0 est au centre)

Angle maxi +/- 1,2 rad, vitesse maxi 11,9 rad/s

Exemple de commande en rotation de la tête : `R09.3/+0.25#` (vitesse 9.3 rad/s, angle 0.25 rad)

C0# ou C1# : déblocage ou blocage du moteur de rotation de la tête (rotation manuelle ou pas)

L0# ou L1# : demande de l'extinction ou de l'allumage du laser

B# : émission d'un bip sonore

Exemple de programme (Communication_serie_python_2.py)

```
import time
from serial import *

# ouverture de la liaison série
ps=Serial(port="COM7",baudrate=115200)
time.sleep(0.1) # pause permettant l'établissement de la liaison série

# écriture de la commande sur la liaison série
ps.write(b"O#") # commande de prise d'origine
ps.write(b"T0.125/0.453#") # commande de translation du chariot
ps.write(b"R09.3/+0.25#") # commande de rotation de la tête
# fermeture de la liaison série
ps.close()
```

Liste des commandes de mesure

P# : demande de la position en mètre. Arduino retourne 6 octets (exemple `0.105;`)

G# : demande de la position angulaire de la tête en radian. Arduino retourne 5 octets (exemple `0.02;`)

D# : demande de la distance du sujet au télémètre en millimètres . Arduino retourne 5 octets (exemple `1209;`)

A# : demande du courant dans le moteur en Ampère. Arduino retourne 5 octets (exemple `0.03;`)

U# : demande de la tension du moteur en volt. Arduino retourne 5 octets (exemple `0.08;`)

I# : demande de la position en incréments du codeur . Arduino retourne un nombre d'octets variable suivant la posiiton.

d# : demande de la distance d'un incrément en mètre. Cet incrément est mesuré lors de la prise d'origine. Il faut donc impérativement exécuter une prise d'origine avant cette commande dans le code Arduino.

Exemple de programme (Communication_serie_python3.py)

```
import time
from serial import *

# ouverture de la liaison série
ps=Serial(port="COM7",baudrate=115200)
time.sleep(0.1) # pause permettant l'établissement de la liaison série
ps.write(b"O#")
reponse=(ps.read(2)) # lecture de la réponse "ok" de l'Arduino
reponse=reponse.decode("utf-8") # transtypage byte -> str
print("Prise d'origine effectuée :",reponse)

ps.write(b"T0.125/0.160#")
time.sleep(4) # pause pour laisser le chariot atteindre sa position

ps.write(b"P#") # demande de la position du chariot
reponse=(ps.read(6)) # lecture de la réponse de l'Arduino
reponse=reponse.decode("utf-8") # transtypage byte -> str
print("Position du chariot :",reponse) # affichage de la réponse

# fermeture de la liaison série
ps.close()
```